DAY 5

Conditional Statements

4-page quick summary | for students new to Python

The big idea: **a condition is a question with a True or False answer.** Python runs the indented block only when the answer is **True**.

`if`	`if-else`	`if-elif-else`	`nested if`
runs only when True	two outcomes	many outcomes	a check inside a check

1 What is a conditional statement?

A conditional statement lets a program **make a decision**. Real-life example: if it is raining, take an umbrella; otherwise, go without one. Python checks whether a condition is **True** or **False**.

- Write a **colon** `:` after the condition.
- **Indent** the statements inside the condition (normally four spaces).
- `=` **assigns** a value; `==` **compares** two values.

Demo 1: a simple if

```
age = 20
if age >= 18:
    print("You are eligible to vote.")
print("Program finished.")
```

```
OUTPUT
You are eligible to vote.
Program finished.
```

Demo 2: a condition that is false

```
temperature = 25
if temperature > 35:
    print("It is very hot.")
print("Check complete.")
```

```
OUTPUT
Check complete.
```

EXPLAIN Demo 1: `20 >= 18` is True, so the message prints. The last `print()` is **outside** the if block, so it always runs. Demo 2: `25 > 35` is False, so Python skips the indented line.

2 Comparison operators

These operators compare values and produce either **True** or **False**.

Operator	Meaning	Example	Result
`==`	equal to	`5 == 5`	`True`
`!=`	not equal to	`5 != 3`	`True`
`>`	greater than	`8 > 4`	`True`
`<`	less than	`2 < 6`	`True`
`>=`	greater than or equal to	`18 >= 18`	`True`
`<=`	less than or equal to	`10 <= 12`	`True`

Demo 3: compare two numbers

```
first_number = 15
second_number = 10
print(first_number == second_number)
print(first_number != second_number)
print(first_number > second_number)
print(first_number < second_number)
print(first_number >= 15)
print(second_number <= 10)
```

```
OUTPUT
False
True
True
False
True
True
```

3 The if-else statement

Use **if-else** when there are **two** possible outcomes: if the condition is True run the **if** block, otherwise run the **else** block.

Demo 5: even or odd

```
number = int(input("Enter an integer: "))
if number % 2 == 0:
    print("Even number")
else:
    print("Odd number")
```

```
OUTPUT
Enter an integer: 7
Odd number
```

Demo 6: student pass or fail

```
marks = float(input("Enter marks out of 100: "))
if marks >= 35:
    print("Result: Pass")
else:
    print("Result: Fail")
```

```
OUTPUT
Enter marks out of 100: 35
Result: Pass
```

EXPLAIN `%` (modulo) returns the **remainder**. A remainder of 0 when dividing by 2 means even; otherwise odd.

Try it: marks **34** gives **Fail**, **35** gives **Pass**, **36** gives **Pass**. 35 passes because `>=` includes the boundary value.

4 The if-elif-else statement

Use this when there are **more than two** outcomes. Python checks the conditions from **top to bottom** and runs the **first match**. If none match, **else** runs.

Demo 7: grade calculator

```
marks = float(input("Enter marks out of 100: "))
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 60:
    print("Grade C")
elif marks >= 35:
    print("Grade D")
else:
    print("Fail")
```

```
OUTPUT
Enter marks out of 100: 80
Grade B
```

Try these marks	95	80	65	40	20
Result	Grade A	Grade B	Grade C	Grade D	Fail

ORDER MATTERS If `marks >= 90` is True, Python does **not** check the later conditions.

Demo 8: classify a number (positive, negative or zero)

```
number = int(input("Enter a number: "))
if number > 0:
    print("Positive")
elif number < 0:
    print("Negative")
else:
    print("Zero")
```

```
OUTPUT
Enter a number: -4
Negative
```

Common beginner mistakes

Mistake	Wrong	Right
= instead of ==	`if age = 18:`	`if age == 18:`
forgetting the colon	`if age >= 18`	`if age >= 18:`
wrong indentation	`body not indented`	`indent the body the same each time`
separate ifs, not elif	`all matching ifs run`	`if-elif-else runs only the first match`
boundary conditions	`> when 18 should pass`	`decide: > or >= ?`
text vs numbers	`input() >= 18`	`int(input()) >= 18`
uppercase and spaces	`answer == "yes" for " YES "`	`answer.strip().lower() == "yes"`

5 Logical operators in conditions

Logical operators **combine** or **reverse** conditions.

Operator	Meaning	Example
`and`	**both** conditions must be True	`age >= 18 and has_id == "yes"`
`or`	**at least one** must be True	`is_student == "yes" or is_senior == "yes"`
`not`	reverses True and False	`not (is_raining == "yes")`

A	B	A and B	A or B	not A
`True`	`True`	`True`	`True`	`False`
`True`	`False`	`False`	`True`	`False`
`False`	`True`	`False`	`True`	`True`
`False`	`False`	`False`	`False`	`True`

Demo 10: age eligibility using and

```
age = int(input("Enter your age: "))
has_id = input("ID card? (yes/no): ").strip().lower()
if age >= 18 and has_id == "yes":
    print("Entry permitted.")
else:
    print("Entry not permitted.")
```

```
OUTPUT
Enter your age: 20
ID card? (yes/no):  YES
Entry permitted.
```

Demo 11: discount eligibility using or

```
is_student = input("Student? (yes/no): ").strip().lower()
is_senior = input("Senior? (yes/no): ").strip().lower()
if is_student == "yes" or is_senior == "yes":
    print("You qualify for a discount.")
else:
    print("Standard price applies.")
```

```
OUTPUT
Student? (yes/no): no
Senior? (yes/no): yes
You qualify for a discount.
```

REMEMBER `.strip()` removes spaces at the start and end, and `.lower()` converts to lowercase, so **YES**, **Yes** and **yes** are treated the same. Demo 12 uses `not`: `if not (is_raining == "yes"):` means "if it is not raining".

6 Nested if statements

A **nested if** is an if inside another if. Use it when the second check should happen **only after** the first condition is satisfied. Each **else** belongs to its nearest matching **if**, based on indentation.

Demo 13: username and password

```
username = input("Enter username: ")
password = input("Enter password: ")
if username == "student":
    if password == "python123":
        print("Login successful.")
    else:
        print("Incorrect password.")
else:
    print("Username not found.")
```

```
OUTPUT
Enter username: student
Enter password: python123
Login successful.

Enter username: student
Enter password: python
Incorrect password.
```

Demo 14: scholarship eligibility

```
marks = float(input("Percentage: "))
family_income = float(input("Annual family income: "))
if marks >= 85:
    if family_income <= 300000:
        print("Eligible to apply for the scholarship.")
    else:
        print("Marks qualify, but income is too high.")
else:
    print("Marks requirement not met.")
```

```
OUTPUT
Percentage: 90
Annual family income: 450000
Marks qualify, but income is too high.
```

NOTE Demo 13 is a classroom demonstration only: real applications must never store passwords as plain text.

7 Practice and recap

More solved examples (Demos 15-20): the key condition in each

Demo	Task	Key condition
15	divisible by 5	`number % 5 == 0`
16	simple calculator	`elif chain on operator; nested if second != 0 for "/"`
17	ticket price	`age < 5 Free; age <= 17 is 100; age <= 59 is 200; else 120`
18	login + verification	`username == "learner" and verification == "5"`
19	valid marks range	`marks < 0 or marks > 100 (invalid), then elif marks >= 35`
20	largest of three	`a >= b and a >= c (ties allowed)`

Practice questions 1-15: the key condition in each solution

Your code may differ and still be right. Compare the **idea**, not the wording. Convert numeric input with int() or float() first.

#	Task	Key condition
1	adult or minor	`age >= 18`
2	even or odd	`number % 2 == 0`
3	pass or fail (40)	`marks >= 40`
4	positive, negative, zero	`number > 0 / elif number < 0 / else`
5	larger of two	`first > second / elif second > first / else equal`
6	grades A to Fail	`>= 90 A, >= 75 B, >= 60 C, >= 40 D, else Fail`
7	divisible by 3 and 5	`number % 3 == 0 and number % 5 == 0`
8	entry: age + ticket	`age >= 18 and ticket == "yes"`
9	member discount (10%)	`member == "yes" then final = price - price * 0.10`
10	month to season	`Winter: month == 12 or month == 1 or month == 2; others use and`
11	username, then PIN	`nested: username == "python_student", then pin == "1234"`
12	between 10 and 50	`number >= 10 and number <= 50`
13	free delivery	`amount >= 500`
14	vowel check	`character == "a" or ... or character == "u" (after .strip().lower())`
15	largest of three	`first >= second and first >= third; elif second ...; else third`

Recap quiz: answers

#	Question	Answer
1	Purpose of an if statement?	runs a block only when its condition is True
2	if-else vs if-elif-else?	two outcomes vs more than two; the first match runs
3	What does == mean, and how is it different from =?	== compares two values; = assigns a value
4	What does and require?	both conditions must be True
5	What does or require?	at least one condition must be True
6	When is a nested if useful?	when a second check should happen only after the first passes
7	Why does indentation matter?	it shows which statements belong to the if, elif or else block
8	What does % return?	the remainder after division
9	Why convert input() with int or float?	input() returns text; numeric comparisons need numbers
10	Which topic is scheduled for Day 6?	for loops, while loops and range()

EXIT TASK Ask for a student's name and marks, print the name, and assign a grade using **if-elif-else**. Explain each line to a partner.

Learning checklist: tick each box when you can do it confidently

- [ ] Write if, if-else and if-elif-else statements with correct colons and indentation.
- [ ] Use comparison operators (==, !=, >, <, >=, <=) to get True or False.
- [ ] Combine conditions with and, or and not.
- [ ] Use a nested if when a second check depends on the first.
- [ ] Explain what input() returns and why int() or float() is needed.